

Universidade Federal de Goiás · Equações Diferenciais Ordinárias

Otimização e Cálculo Numérico de Autovalores

Algoritmo geral (Hessenberg + QR), escalabilidade, otimização simétrica e aplicação em reatores

Antonio Couto · 202400305

Miguel Bertol · 202400315

Implementação em C++17

Roteiro

- **Fase 1** — O algoritmo geral: Householder → Hessenberg → iteração QR
- **Fase 2** — Escalabilidade empírica (tempo e memória)
- **Fase 3** — Otimização para matrizes simétricas (colapso tridiagonal)
- **Fase 4** — Aplicação: rede de reatores químicos (CSTRs)
- **Discussão** — Por que a simetria acelera tanto? CPU ou memória?

Introdução

Sistema para cálculo numérico de autovalores de matrizes quadradas genéricas, desenvolvido em **C++17**.

Teorema de Abel-Ruffini: para matrizes de ordem $N \geq 5$, *não existe fórmula fechada* para as raízes do polinômio característico.

⇒ Métodos **iterativos** são a única alternativa viável.

Arquitetura do projeto

Estruturas centrais

- `Matrix` — $N \times N$ densa, row-major
- `Vector` — produto escalar, norma L_2
- `SymmetricTridiagonal` — só `diag` e `offdiag`

Princípios

- Semântica de **valor** (sem mutação in-place)
- Memória delegada ao `std::vector`
- Multiplicação em ordem i - k - j (localidade de cache)

A estrutura `SymmetricTridiagonal`

Armazena apenas **dois vetores** em vez de uma matriz densa:

- `diag` — N entradas (diagonal principal)
- `offdiag` — $N - 1$ entradas (sub/superdiagonal)

Representação compacta de $\mathbf{O}(\mathbf{N})$ em vez de $O(N^2)$ — central para a otimização da **Fase 3**.

FASE 1

O Algoritmo Geral

Householder $\rightarrow$ Hessenberg $\rightarrow$ Iteração QR com shift de Wilkinson

Transformações de Householder

Refletor ortogonal: $P = I - 2\mathbf{v}\mathbf{v}^T$

Dado $\mathbf{x} \in \mathbb{R}^m$, escolhemos $\mathbf{v}$ para zerar tudo menos a 1ª componente:

$$P\mathbf{x} = -\text{sign}(x_0) \|\mathbf{x}\|_2 \cdot \mathbf{e}_1$$

Construção de $\mathbf{v}$ — estabilidade

$$\mathbf{v} = \frac{\mathbf{x} + \text{sign}(x_0) \|\mathbf{x}\| \mathbf{e}_1}{\|\mathbf{x} + \text{sign}(x_0) \|\mathbf{x}\| \mathbf{e}_1\|}$$

A escolha do sinal $\text{sign}(x_0)$ **evita cancelamento catastrófico**:
somamos na direção do componente dominante, e o denominador
nunca se aproxima de zero.

```
1 // householder.cpp
2 double sign = (x[0] >= 0.0) ? 1.0 : -1.0;
3 v[0] = x[0] + sign * x_norm; // soma na direção dominante
4 for (std::size_t i = 1; i < len; ++i) v[i] = x[i];
5 double v_norm = v.norm();
6 for (std::size_t i = 0; i < len; ++i) v[i] /= v_norm;
```


Aplicação implícita — sem montar P

Para PA (pela esquerda), por coluna j : projeção $p = \mathbf{v}^T \mathbf{a}_j$, depois $a_{ij} \leftarrow a_{ij} - 2v_i p$.

```
// apply_householder_left — custo  $O(m)$  por coluna, nunca forma  $P$ 
for (std::size_t col = col_start; col < col_end; ++col) {
    double projection = 0.0;
    for (std::size_t i = 0; i < len; ++i)
        projection += v[i] * A(row_start + i, col);
    for (std::size_t i = 0; i < len; ++i)
        A(row_start + i, col) -= 2.0 * v[i] * projection;
}
```

A aplicação pela direita (AP) é a lógica transposta, sobre linhas.

Redução à forma de Hessenberg

Hessenberg superior: zera tudo abaixo da 1ª subdiagonal.

$$H(i, j) = 0 \quad \text{para } i > j + 1$$

Reduzimos A por **transformações de similaridade** $A \leftarrow P_k A P_k^T$,
que *preservam os autovalores*.

$N - 2$ passos

custo $O(N^3)$

Hessenberg — passo k

1. Extrair a subcoluna $\mathbf{x} = A(k+1:N-1, k)$
2. Computar o refletor $\mathbf{v}$ a partir de $\mathbf{x}$
3. Aplicar a similaridade $A \leftarrow P_k A P_k^T$ (esquerda zera a subcoluna, direita mantém a similaridade)

Após $N - 2$ passos: H em forma de Hessenberg, mesmos autovalores de A . Custo $O(N^3)$, dominado pelos produtos matriz-refletor.

Iteração QR

Dada H_k , fatoramos $H_k = Q_k R_k$ e recombinaamos:

$$H_{k+1} = R_k Q_k = Q_k^T H_k Q_k$$

Mesma similaridade $\implies$ **autovalores preservados**; sob certas condições a subdiagonal converge a zero, revelando os autovalores na diagonal.

Fatoração QR via rotações de Givens

$N - 1$ rotações, cada uma zerando um elemento da subdiagonal:

$$\begin{bmatrix} c & -s \\ s & c \end{bmatrix} \begin{bmatrix} h_{ii} \\ h_{i+1,i} \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix}$$

Coeficientes com **branching numérico**: se $|h_{i+1,i}| > |h_{ii}|$, usa-se $\tau = -h_{ii}/h_{i+1,i}$ para evitar *overflow*.

Por que precisamos do shift?

além do enunciado O enunciado (§1.2) pede QR **puro**: $H = QR$, $H_{k+1} = R_k Q_k$, iterar até convergir.

Mas o QR puro converge só **linearmente**, à taxa $|\lambda_{i+1}/\lambda_i|$. Com autovalores de magnitude próxima (comuns nas matrizes aleatórias da Fase 2) a razão $\rightarrow 1$ e a convergência **estagna**.

No limite — autovalores de igual magnitude, ex. pares $\pm\lambda$ — a iteração não-deslocada **nunca converge**.

$\Rightarrow$ Sem shift, o próprio benchmark da Fase 2 não terminaria. O shift não é refinamento: é **necessário** para a corretude prática.

Deslocamento de Wilkinson

Acelera a convergência escolhendo um shift μ a cada iteração:

$$\mu = \text{autovalor de } \begin{bmatrix} h_{n-1,n-1} & h_{n-1,n} \\ h_{n,n-1} & h_{n,n} \end{bmatrix} \text{ mais próximo de } h_{n,n}$$

Aplica QR a $(H - \mu I)$ e recupera o shift $\Rightarrow h_{n,n-1}$ converge **cubicamente** para zero.

Deflação

Quando a subdiagonal é desprezível (critério relativo, $\varepsilon = 10^{-10}$):

$$|h_{i+1,i}| < \varepsilon (|h_{ii}| + |h_{i+1,i+1}|)$$

- Bloco $1 \times 1 \implies$ autovalor **real**
- Bloco 2×2 com discriminante negativo $\implies$ par **conjugado complexo**

Custo: $O(N^2)$ por iteração QR $\cdot O(N)$ iterações $\implies O(N^3)$ na fase QR.

FASE 2

Escalabilidade do Algoritmo Geral

Benchmark: $N \in \{10, 50, 100, 250, 500, 1000\} \cdot 20$ amostras por tamanho

Ambiente experimental

Hardware

- CPU: Intel Core **i5-13450HX**
- 10 núcleos / 16 threads · até 4,6 GHz
- Cache L1d 416 KiB · L2 9,5 MiB · **L3 20 MiB**
- RAM: **16 GB**

Software

- SO: **Ubuntu 24.04 LTS**
- Compilador: **g++ 13.3.0**
- Flags: `-std=c++17 -O2`

As bandas $O(N)$ da tridiagonal (Fase 3) cabem confortavelmente em cache L1/L2 para todos os N testados.

Tempo — algoritmo geral (mediana de 20 amostras)

N	tempo (ms)
10	0,02
50	1,09
100	7,55
250	106,4
500	823,2
1000	6685,6

Ajuste empírico: $T(N) \sim O(N^{2,76})$ —
compatível com $O(N^3)$ teórico

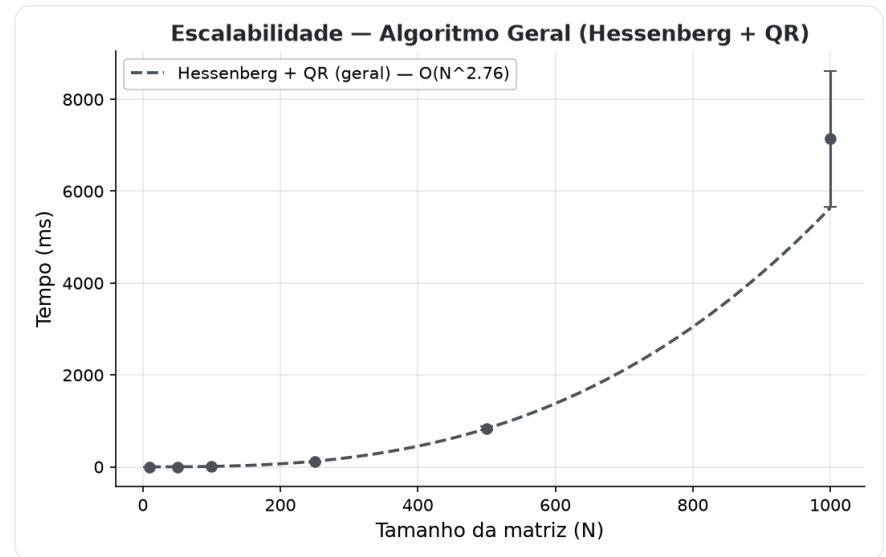


gráfico: média $\pm$ desvio

Memória RSS — algoritmo geral (mediana de 20 amostras)

N	memória (KB)
10	0 (< granul.)
50	40
100	156
250	976
500	3908
1000	15636

$M(N) \sim O(N^2)$ — aloca duas matrizes densas $N \times N$ ($2N^2$ doubles)

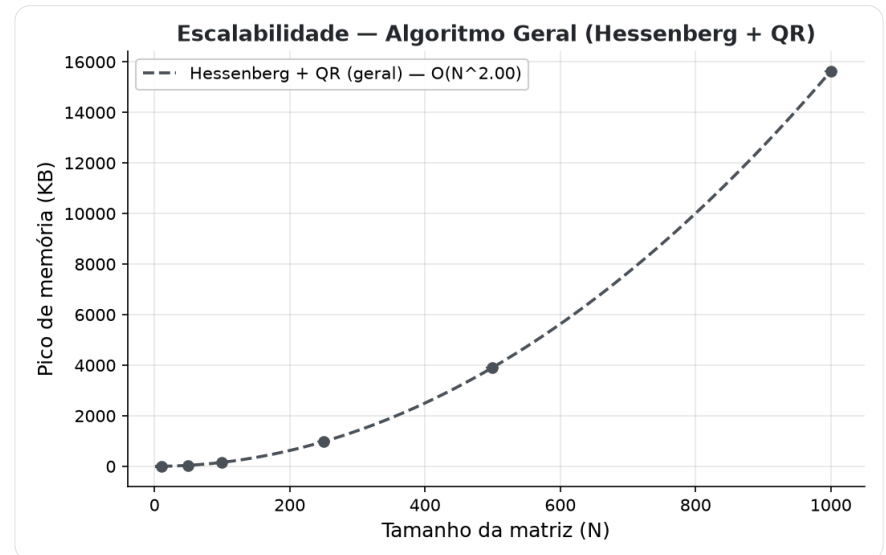


gráfico: média $\pm$ desvio

FASE 3

Otimização para Matrizes Simétricas

$A = A^T \Rightarrow$ o problema colapsa para tridiagonal

O colapso tridiagonal

Se $A = A^T$, a redução de Householder produz uma **tridiagonal simétrica**.

Prova. Cada $P_k = P_k^T = P_k^{-1}$. A similaridade preserva a simetria:

$$(P_k A P_k^T)^T = P_k A^T P_k^T = P_k A P_k^T$$

Simétrica e Hessenberg (nula abaixo da subdiagonal) $\implies$ só sobra a diagonal e as duas adjacentes $\implies$ **tridiagonal**.

Demonstração empírica ($N = 5$)

O programa aplica o algoritmo geral de Hessenberg a uma matriz simétrica e verifica que a maior entrada **fora da banda tridiagonal** é:

$$\sim 10^{-16} \quad (\text{precisão de máquina})$$

Ou seja: o colapso não é só teórico — acontece até o último bit.

Redução tridiagonal otimizada

Atualização simétrica de **rank-2** no bloco $B = A(k+1:, k+1:)$:

$$\mathbf{p} = 2B\mathbf{v}$$

$$\mathbf{w} = \mathbf{p} - (\mathbf{v}^T \mathbf{p})\mathbf{v}$$

$$B \leftarrow B - \mathbf{w}\mathbf{v}^T - \mathbf{v}\mathbf{w}^T$$

```
1 // tridiagonal.cpp - rank-2 update
2 for (std::size_t i = 0; i < m; ++i) {
3     double sum = 0.0;
4     for (std::size_t j = 0; j < m; ++j)
5         sum += A(col+1+i, col+1+j) * v[j];
6     p[i] = 2.0 * sum;           // p = 2 B v
7 }
8 // w = p - (v^T p) v ; B -= w v^T + v w^T
```

Preserva a simetria algebricamente; estado próprio em $O(N)$.

Iteração QL implícita (tqli)

QR sobre a forma tridiagonal, operando só nos dois vetores d (diag) e e (subdiag). Para cada índice l :

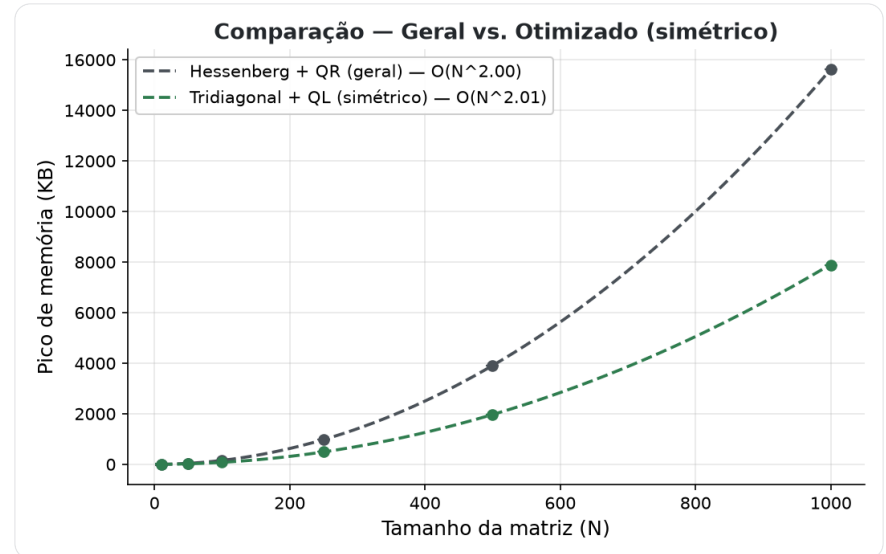
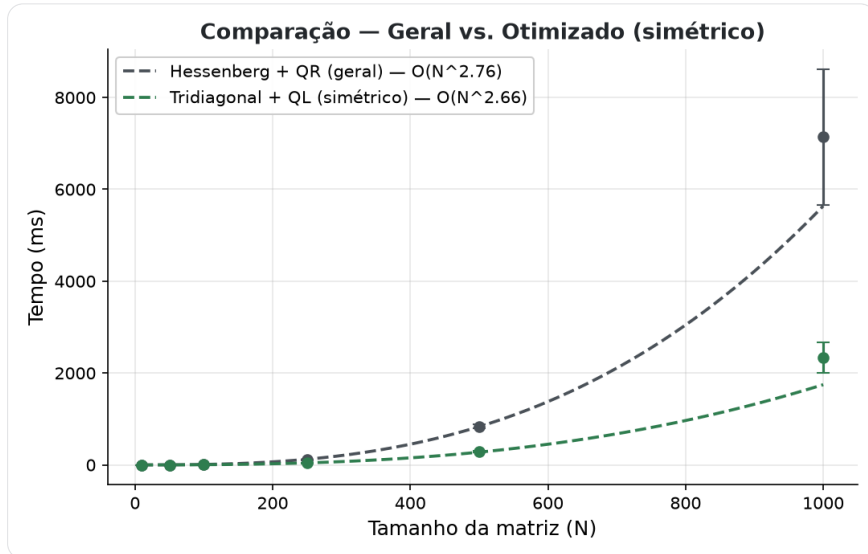
1. Achar o menor $m > l$ com $|e_m| \leq \varepsilon(|d_m| + |d_{m+1}|)$

2. Shift de Wilkinson: $g = \frac{d_{l+1} - d_l}{2e_l}$, $g = d_m - d_l + \frac{e_l}{g + \text{sign}(g)\sqrt{g^2 + 1}}$

3. Cadeia de rotações de Givens de $m - 1$ até l , atualizando só d e e — cada rotação $O(1)$, varredura $O(N)$

$O(N)$ por iteração $\cdot O(N)$ iterações $\Rightarrow O(N^2)$ total (vs. $O(N^3)$ do geral).

Comparação — agora sobrepomos os dois



Mesmos eixos da Fase 2, agora com o caminho simétrico (verde) sobreposto — a separação cresce com N .

gráficos: média $\pm$ desvio de 20 amostras

Resultados comparativos (ganhos pela mediana de 20 amostras)

N	ganho tempo	ganho memória
10	50%	—
50	61%	60%
100	64%	51%
250	65%	49%
500	65%	50%
1000	67%	50%

- Tempo: $O(N^{2,66})$ vs $O(N^{2,76})$
- Memória: ambos $O(N^2)$, coef. tri $\approx$ **metade**
- Ganho de tempo **cresce** com N (efeito de cache)

FASE 4

Rede de Reatores (CSTRs)

$N = 100$ reatores de mistura perfeita em série

O modelo

Dinâmica de concentrações por um sistema linear de EDOs:

$$\frac{d\mathbf{C}}{dt} = A\mathbf{C}$$

A é tridiagonal simétrica:

- Diagonal $-2,5$ — perda por reação e escoamento
- Sub/superdiagonais $+1,0$ — troca difusiva entre vizinhos

Construída diretamente como `SymmetricTridiagonal` $\implies$ caminho otimizado.

Análise 1 — Estabilidade

Fórmula analítica dos autovalores (tridiagonal simétrica de Toeplitz):

$$\lambda_k = -2,5 + 2 \cos\left(\frac{k\pi}{N+1}\right)$$

Máximo: $\lambda_{\max} = -2,5 + 2 \cos\left(\frac{\pi}{101}\right) \approx -0,50$

Todos os autovalores têm $\text{Re} < 0 \implies$ sistema **assintoticamente estável**: as concentrações convergem ao estado estacionário.

Análise 2 — Rigidez (*stiffness*)

$$S = \frac{\max |\operatorname{Re}(\lambda_i)|}{\min |\operatorname{Re}(\lambda_i)|} = \frac{|\lambda_{\min}|}{|\lambda_{\max}|} \approx \frac{4,50}{0,50} \approx 9$$

$S \approx 9$ é **baixo** (muito abaixo do limiar 10^3) $\Rightarrow$ sistema **não-rígido**.

Consequência de engenharia: pode-se usar com segurança um solver **explícito** (RK4), evitando o custo de métodos implícitos.

Por que a simetria acelera tanto?

Três efeitos acumulados, todos derivados de $A = A^T$:

1. **Redução ao tridiagonal** — representação cai de $O(N^2)$ para $O(N)$; cada operação toca muito menos dado.
2. **Queda de complexidade do QR** — cada rotação afeta $O(1)$ entradas (não $O(N)$ colunas) $\Rightarrow O(N^3) \rightarrow O(N^2)$.
3. **Localidade de cache** — dois vetores contíguos cabem em L1/L2; o caminho denso percorre matrizes com *stride* variável. Amplifica o ganho para $N \geq 250$.

O gargalo é CPU ou memória?

Resposta: **CPU**.

- Ambos escalam $O(N^2)$ em memória
- Ganho de **tempo** (67% em $N = 1000$) supera o de **memória** (50%)

Se memória fosse o limite, os ganhos seriam proporcionais. A diferença mostra que a **redução de operações de ponto flutuante** — não a economia de alocação — é o fator dominante.

Conclusão

- **Fase 1** — algoritmo geral Hessenberg + QR (Wilkinson), $O(N^3)$
- **Fase 2** — empírico: $T \sim O(N^{2,76})$, $M \sim O(N^2)$
- **Fase 3** — simetria $\Rightarrow$ tridiagonal $\Rightarrow O(N^2)$, até 67% mais rápido
- **Fase 4** — reatores estáveis e não-rígidos $\Rightarrow$ RK4 explícito

Explorar a estrutura do problema vale mais que força bruta.